Data Pipeline

Source Data

The source match files are MKW Table Bot generated JSON files. In this repo, many are saved with a `.txt` extension even though their contents are JSON.

Example folders:

- `backend/JSON/ctc_s1_d1_2/`
- `backend/JSON/ctc_s1_d3/`
- `backend/JSON/ctc_s1_d4/`

Expected Input Shape

`backend/extract.py` expects each file to contain:

- `tracks`: ordered list of track names
- `teams`: object keyed by team name
- each team has `players`
- each player has:
  - `race_scores`
  - `lounge_name`
  - `mii_name`

The extractor uses `lounge_name` if present, otherwise `mii_name`.

Extraction Logic

`extract.extract(foldername, outputdirectory)`:

1. Finds every `*.txt` file in `BASE_DIR / foldername`.
2. Parses the file as JSON.
3. Loops over teams.
4. Loops over players in each team.
5. Loops over each player's `race_scores`.
6. Writes one CSV row per race with:
   - team
   - player
   - track
   - score

The output header is:

```
team,player,track,score
```

Bagging Marker

If a player's race score is `1`, the extractor appends `(bag)` to the player name:

```
player_name + " (bag)"
```

This means bagging versions are treated as separate players in the analytics.

Current CSV Outputs

Current active Season 1 CSV files:

- `backend/CSV/ctc_d1_2.csv`
- `backend/CSV/ctc_d3.csv`
- `backend/CSV/ctc_d4.csv`

Other CSV files exist:

- `backend/CSV/ctc_d1_2 (teams renamed).csv`
- `backend/CSV/rt_gsc_s13.csv`

The Flask app only reads files matching `ctc_d{division}.csv`.

Manual Rebuild Example

The extraction function is not exposed as a command-line interface. To rebuild a CSV today, you would need to call it from Python, for example:

```
from extract import extract

extract("JSON/ctc_s1_d4", "CSV/ctc_d4.csv")
```

Because `outputdirectory` is passed directly to `pandas.DataFrame.to_csv`, relative paths depend on the current working directory. This should be made safer before relying on it for Season 3 maintenance.

Current Maintenance Workflow

The current practical workflow is:

1. Add new MKW Table Bot JSON/text files under a season/division folder.
2. Run extraction manually to regenerate the division CSV.
3. Restart or wait out the Flask cache if the API is already running.
4. Redeploy if the data is committed into the repo.

Gaps For Live Season 3

- No upload endpoint.
- No validation report for malformed match files.

- No duplicate detection for repeated matches.
- No player/team alias system.
- No season field in the flattened CSV.
- No automatic cache invalidation.
- No audit trail showing when a match was added.
- No script that rebuilds every season/division deterministically.

Recommended Data Direction

For Season 2 and 3, make season and division explicit fields instead of encoding them only in filenames:

```
season,division,week,match_id,team,player,track,score
```

Additional useful fields:

- source filename
- table bot match id or thread id
- opponent team
- race number
- player friend code, if stable enough to use
- raw lounge name
- raw mii name
- normalized player name
- normalized team tag
- is_bagging_row

This would let one API serve all seasons and make uploads incremental.